

LAPORAN PRAKTIKUM
STRUKTUR DATA DAN ALGORITMA
ANALISIS KOMPLEKSITAS ALGORITMA



Disusun Oleh:

Nama : Muhammad Ziyen Rizki
NIM : 2025573010086
Kelas : TI-1C
Dosen : Muhammad Reza Zulman, SST., M..Sc

PRODI TEKNIK INFORMATIKA
TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
TAHUN AJARAN 2025/2026

A. Tujuan Praktikum

- Memahami mengapa analisis efisiensi algoritma itu penting
- Menjelaskan konsep Time Complexity dan Space Complexity
- Membaca dan menulis notasi Big O: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$
- Menganalisis kompleksitas loop tunggal dan nested loop
- Membandingkan efisiensi dua algoritma secara empiris menggunakan `Date.now()`
- Mengenali pola kode yang memiliki kompleksitas buruk dan cara memperbaikinya

B. Alat dan Bahan

- Laptop
- Visual Code Studio
- Node.js
- Terminal
- Modul 5

C. Langkah Langkah Praktikum

1. Buat folder baru dengan nama praktikum-5
2. Lakukan deklarasi menggunakan command ***npm init -y***. pastikan package.json sudah ada di dalamnya
3. Masuk kedalam folder praktikum-5, kemudian lakukan pembuatan file baru dengan nama 01-intro-kompleksitas.js
4. Masuk kedalam file 01-intro-kompleksitas.js. Kemudian masukkan code berikut

```
function ukurWaktu(label, fn) {
  const mulai = Date.now();
  fn();
  const selesai = Date.now();
  console.log(`${label}: ${selesai - mulai} ms`);
}
const N = 100_00;
function jumlahkanLinear(n) {
  let total = 0;
  for (let i = 1; i <= n; i++) total += i;
  return total;
}
function jumlahkanRumus(n) {
  return (n * (n + 1)) / 2;
}
function CariPasangan(arr){
```

```

const pasangan = [];
for(let i = 0; i < arr.length; i++){
  for(let j = i + 1; j < arr.length; j++){
    if(arr[i] + arr[j] === 10)
      pasangan.push([arr[i], arr[j]]);
  }
}
return pasangan;
}
const data = Array.from({ length: 5000 }, (_, i) => i);

console.log ('=== perbandingan waktu esksekusi ===');
ukurWaktu ('O(1) jumlahkanRumus ', () => jumlahkanRumus(N));
ukurWaktu ('O(n) jumlahkanLinear ', () => jumlahkanLinear(N));
ukurWaktu ('O(n^2) CariPasangan ', () => CariPasangan(data));

console.log ('\nHasil sama?', jumlahkanLinear (100) === jumlahkanRumus(100));

```

5. Buat file baru dengan nama 02-kelas-big-o.js. Masuk kedalam file 02-kelas-big-o.js kemudian masukkan code

```

function ambilPertama(arr) {
  return arr[0];
}

function tambahkanItem(arr, item) {
  arr.push(item);
}

function isGenap(n) {
  return n % 2 === 0;
}

function binarySearch(arr, target) {
  let kiri = 0, kanan = arr.length - 1;
  let langkah = 0;
  while (kiri <= kanan) {
    langkah++;
    const tengah = Math.floor((kiri + kanan) / 2);
    if (arr[tengah] === target) {
      console.log(`Ditemukan di indeks ${tengah} setelah ${langkah}
langkah`);
      return tengah;
    }
    if (arr[tengah] < target) kiri = tengah + 1;
    else kanan = tengah - 1;
  }
  return -1;
}

function cariMax(arr) {
  let maks = arr[0];
  for (const x of arr) if (x > maks) maks = x;
  return maks;
}

```

```

function bubbleSortNaif(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length; i++)
    for (let j = 0; j < a.length - i - 1; j++)
      if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]];
  return a;
}

function fibRekursif(n) {
  if (n <= 1) return n;
  return fibRekursif(n - 1) + fibRekursif(n - 2);
}

const memo = {};
function fibMemo(n) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
  return memo[n] = fibMemo(n - 1) + fibMemo(n - 2);
}

console.log('=== O(1) ===');
console.log(ambilPertama([10, 20, 30, 40, 50]));
console.log(isGenap(42));

console.log('\n=== O(log n) ===');
const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
binarySearch(sorted, 731_452);

console.log('\n=== O(n) ===');
console.log('Max:', cariMax(Array.from({ length: 1000 }, () => Math.random() * 1000 | 0)));

console.log('\n=== O(n^2) ===');
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));

console.log('\n=== O(2^n) ===');
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + ' ');
console.log('');

console.log('\nWaktu fib(35) O(2^n):');
let t = Date.now();
fibRekursif(35);
console.log(Date.now() - t, 'ms');

console.log('Waktu fib(35) O(n) memoization:');
t = Date.now();
fibMemo(35);
console.log(Date.now() - t, 'ms');

```

6. Buat file baru dengan nama 03-space-complexity.js. Masuk kedalam file 03-space-complexity.js, kemudian masukkan code

```

function jumlahArray(arr) {
  let total = 0;
  for (const x of arr) total += x;
}

```

```

    return total;
}

function duplikasiArray(arr) {
    const baru = [];
    for (const x of arr) baru.push(x * 2);
    return baru;
}

function faktorialRekursif(n) {
    if (n <= 1) return 1;
    return n * faktorialRekursif(n - 1);
}

function faktorialIteratif(n) {
    let hasil = 1;
    for (let i = 2; i <= n; i++) hasil *= i;
    return hasil;
}

function hitungUnik(arr) {
    const seen = new Set();
    for (const x of arr) seen.add(x);
    return seen.size;
}

const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log('Jumlah array :', jumlahArray(arr));
console.log('Duplikasi array:', duplikasiArray(arr));
console.log('Faktorial 10 rekursif :', faktorialRekursif(10));
console.log('Faktorial 10 iteratif :', faktorialIteratif(10));

const dataAcak = [1, 2, 3, 2, 1, 4, 5, 3, 6, 4, 7];
console.log('Elemen unik:', hitungUnik(dataAcak));

console.log('\n\n===== Latihan Class BankAccount =====');
function cariPasanganLambat(arr, target) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (arr[i] + arr[j] === target) {
                return [arr[i], arr[j]];
            }
        }
    }
    return null;
}

function cariPasanganCepat(arr, target) {
    const seen = new Set();
    for (let i = 0; i < arr.length; i++) {
        let complement = target - arr[i];
        if (seen.has(complement)) {
            return [complement, arr[i]];
        }
        seen.add(arr[i]);
    }
    return null;
}

```

```

const dataUji = [2, 7, 11, 15];
const targetUji = 9;

console.log("--- Uji Fungsi ---");
console.log("Lambat:", cariPasanganLambat(dataUji, targetUji));
console.log("Cepat :", cariPasanganCepat(dataUji, targetUji));

const n = 50000;
const dataBesar = Array.from({ length: n }, (_, i) => i);
const targetBesar = -1;

console.log("\n--- Benchmark (n=50.000) ---");

let t1 = performance.now();
cariPasanganLambat(dataBesar, targetBesar);
let s1 = performance.now();
console.log(`Waktu Lambat O(n^2): ${s1 - t1}.toFixed(4)} ms`);

let t2 = performance.now();
cariPasanganCepat(dataBesar, targetBesar);
let s2 = performance.now();
console.log(`Waktu Cepat O(n) : ${s2 - t2}.toFixed(4)} ms`);

```

7. Buat file baru dengan nama 04-refactor.js. Masuk kedalam file 04-refactor.js, kemudian masukkan code berikut

```

function adaDuplikatLambat(arr) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] === arr[j]) return true;
    }
  }
  return false;
}

function adaDuplikatCepat(arr) {
  const seen = new Set();
  for (const x of arr) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
  return false;
}

function frekuensiLambat(arr) {
  const unik = [];
  const hitung = [];
  for (const x of arr) {
    const idx = unik.indexOf(x);
    if (idx === -1) {
      unik.push(x);
      hitung.push(1);
    } else {
      hitung[idx]++;
    }
  }
}

```

```

    return Object.fromEntries(unik.map((u, i) => [u, hitung[i]]));
}

function frekuensiCepat(arr) {
    const counter = {};
    for (const x of arr) {
        counter[x] = (counter[x] || 0) + 1;
    }
    return counter;
}

const besar = Array.from({ length: 20000 }, () => Math.floor(Math.random() * 1000));

let t = Date.now();
adaDuplikatLambat(besar);
console.log('Duplikat LAMBAT O(n2):', Date.now() - t, 'ms');

t = Date.now();
adaDuplikatCepat(besar);
console.log('Duplikat CEPAT O(n) :', Date.now() - t, 'ms');

const kata = ['a', 'b', 'a', 'c', 'b', 'a', 'd'];
console.log('\nFrekuensi:', frekuensiCepat(kata));

```

8. Buat file dengan nama latihan-1.js. Untuk mengerjakan soal latihan yang tersedia pada modul Latihan 1: Identifikasi Kompleksitas. Masuk kedalam file latihan-1.js, kemudian masukkan code

```

function hitungKompleksitas(n, fn) {
    const mulai = Date.now();
    fn(n);
    const selesai = Date.now();
    console.log(`Waktu eksekusi: ${selesai - mulai} ms`);
}

function fnA(n) {
    return n * 2;
}

function fnB(n) {
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            console.log(i, j);
        }
    }
}

function fnC(n) {
    for (let i = 1; i < n; i *= 2) {
        console.log(i);
    }
}

function fnD(n) {
    const arr = Array.from({ length: n }, (_, i) => i);
}

```

```

arr.forEach(x => {
  arr.forEach(y => {
    arr.forEach(z => {
      console.log(x, y, z);
    });
  });
});
}

```

```
const n = 1000;
```

```
hitungKompleksitas(n, fnA);
hitungKompleksitas(n, fnC);
```

9. Buat folder baru didalam folder praktikum 5 dengan nama folder *tugas*. Masuk kedalam folder tugas untuk membuat dua file baru, yakni tugas-1.js dan tugas-2.js

10. Masuk kedalam file tugas-1.js untuk mengerjakan tugas 1 pada modul yg berjudul Tugas

1: Analisis dan Refactor. Masukkan code

```

function intersectionLambat(arr1, arr2) {
  const hasil = [];
  for (let i = 0; i < arr1.length; i++) {
    if (arr2.includes(arr1[i]) && !hasil.includes(arr1[i])) {
      hasil.push(arr1[i]);
    }
  }
  return hasil;
}

function intersectionCepat(arr1, arr2) {
  const set2 = new Set(arr2);
  return [...new Set(arr1.filter(x => set2.has(x)))];
}

function groupAnagrams(strs) {
  const map = {};
  for (let s of strs) {
    let key = s.split('').sort().join('');
    if (!map[key]) map[key] = [];
    map[key].push(s);
  }
  return Object.values(map);
}

function cekKuadratLambat(arr) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = 0; j < arr.length; j++) {
      for (let k = 0; k < arr.length; k++) {
        if (i !== j && arr[i] + arr[j] === arr[k] ** 2) return true;
      }
    }
  }
  return false;
}

```



```

function cekKuadratCepat(arr) {
    const setKuadrat = new Set(arr.map(x => x ** 2));
    arr.sort((a, b) => a - b);
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (setKuadrat.has(arr[i] + arr[j])) return true;
        }
    }
    return false;
}

const n = 2000;
const dataA1 = Array.from({ length: n }, (_, i) => i);
const dataA2 = Array.from({ length: n }, (_, i) => i + 500);
const dataC = Array.from({ length: 500 }, (_, i) => i + 1);

console.log("=== Fungsi A (Intersection) ===");
let t = performance.now();
intersectionLambat(dataA1, dataA2);
console.log(`Lambat: ${performance.now() - t}.toFixed(4)} ms`);

t = performance.now();
intersectionCepat(dataA1, dataA2);
console.log(`Cepat: ${performance.now() - t}.toFixed(4)} ms`);

console.log("\n=== Fungsi B (Anagrams) ===");
console.log(groupAnagrams(['eat', 'tea', 'tan', 'ate', 'nat', 'bat']));

console.log("\n=== Fungsi C (a + b = c^2) ===");
t = performance.now();
cekKuadratLambat(dataC);
console.log(`Lambat: ${performance.now() - t}.toFixed(4)} ms`);

t = performance.now();
cekKuadratCepat(dataC);
console.log(`Cepat: ${performance.now() - t}.toFixed(4)} ms`);

```

11. Masuk kedalam file tugas-2.js untuk mengerjakan tugas 2 pada modul yg berjudul Visualisasi Pertumbuhan Big O. Masukkan code

```

function fn_01(n) {
    return n + 1;
}

function fn_On(n) {
    let total = 0;
    for (let i = 0; i < n; i++) {
        total += i;
    }
    return total;
}

function fn_OnLogn(n) {
    let total = 0;
    for (let i = 0; i < n; i++) {

```

```

        for (let j = 1; j < n; j *= 2) {
            total += i + j;
        }
    }
    return total;
}

function fn_On2(n) {
    let total = 0;
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            total += i + j;
        }
    }
    return total;
}

function benchmarkSemua(ukuranData) {
    console.log("N\t | O(1)\t | O(n)\t | O(n log n)\t | O(n^2)");
    console.log("-----");

    ukuranData.forEach(n => {
        const t0 = performance.now();
        fn_O1(n);
        const d0 = (performance.now() - t0).toFixed(4);

        const t1 = performance.now();
        fn_On(n);
        const d1 = (performance.now() - t1).toFixed(4);

        const t2 = performance.now();
        fn_OnLogn(n);
        const d2 = (performance.now() - t2).toFixed(4);

        const t3 = performance.now();
        fn_On2(n);
        const d3 = (performance.now() - t3).toFixed(4);

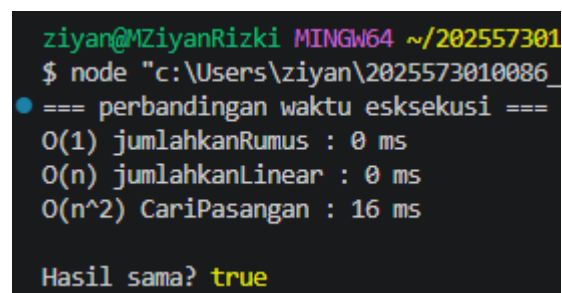
        console.log(`${n}\t | ${d0}\t | ${d1}\t | ${d2}\t | ${d3} ms`);
    });
}

benchmarkSemua([100, 500, 1000, 5000, 10000]);

```

D. Hasil Praktikum

1. 01-into-kompleksitas.js



```

ziyan@MziyanRizki MINGW64 ~/202557301
$ node "c:\Users\ziyan\2025573010086_
=== perbandingan waktu eskekusi ===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 0 ms
O(n^2) CariPasangan : 16 ms

Hasil sama? true

```

2. 02-kelas-big-o.js

```
ziyan@MziyanRizki MINGW64 ~/2025573010086_Praktikum-S
$ node "c:\Users\ziyan\2025573010086_Praktikum-S
=== O(1) ===
10
true

=== O(log n) ===
Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) ===
Max: 998

=== O(n^2) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2^n) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2^n):
75 ms
Waktu fib(35) O(n) memoization:
0 ms
```

3. 03-space-complexity.js

```
ziyan@MziyanRizki MINGW64 ~/2025573010086_Pra
$ node "c:\Users\ziyan\2025573010086_Praktiku
Jumlah array : 55
Duplikasi array: [
  2, 4, 6, 8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7

===== Latihan Class BankAccount =====
--- Uji Fungsi ---
Lambat: [ 2, 7 ]
Cepat : [ 2, 7 ]

--- Benchmark (n=50.000) ---
Waktu Lambat O(n^2): 950.0874 ms
Waktu Cepat O(n) : 4.5655 ms
```

4. 04-refactor.js

```
ziyan@MziyanRizki MINGW64 ~/2025573010086_Praktikum-Struktur-Da
$ node "c:\Users\ziyan\2025573010086_Praktikum-Struktur-Da
Duplikat LAMBAT O(n2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

5. latihan-1.js

```
ziyan@MziyanRizki MINGW64 ~/2025573010086_Praktikum-Struktur-Da
$ node "c:\Users\ziyan\2025573010086_Praktikum-Struktur-Da
Waktu eksekusi: 0 ms
1
2
4
8
16
32
64
128
256
512
Waktu eksekusi: 2 ms
```

6. tugas/tugas-1.js

```
ziyan@MziyanRizki MINGW64 ~/2025573010086_Praktikum-Struktur-Da
$ node "c:\Users\ziyan\2025573010086_Praktikum-Struktur-Da
=== Fungsi A (Intersection) ===
Lambat: 2.8429 ms
Cepat: 0.2421 ms

=== Fungsi B (Anagrams) ===
[ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]

=== Fungsi C (a + b = c^2) ===
Lambat: 0.0681 ms
Cepat: 0.1246 ms
```

7. tugas/tugas-1.js

```
ziyan@MziyanRizki MINGW64 ~/2025573010086_Praktikum-Struktur-Da-Algoritma (main)
$ node "c:\Users\ziyan\2025573010086_Praktikum-Struktur-Da-Algoritma\praktikum-5
N      | O(1) | O(n) | O(n log n) | O(n^2)
-----|-----|-----|-----|-----
100    | 0.0087 | 0.0193 | 0.0212 | 0.2106 ms
500    | 0.0138 | 0.0059 | 0.0837 | 0.9817 ms
1000   | 0.0006 | 0.0152 | 0.0259 | 0.8727 ms
5000   | 0.0393 | 0.0059 | 0.1038 | 34.1063 ms
10000  | 0.0013 | 0.0074 | 0.1634 | 130.0625 ms
```

E. Kesimpulan

Modul ini menekankan pentingnya analisis efisiensi melalui notasi Big O untuk mengukur laju pertumbuhan waktu dan memori suatu algoritma seiring bertambahnya data input. Mahasiswa diajarkan untuk mengidentifikasi berbagai kelas kompleksitas, mulai dari $O(1)$ yang konstan hingga $O(2^n)$ yang eksponensial, serta memahami konsep *space complexity* yang berkaitan dengan penggunaan memori. Salah satu poin krusial dalam praktikum ini adalah teknik optimasi kode, seperti melakukan refaktor pada pola *nested loop* yang lambat ($O(n^2)$) menjadi lebih efisien menggunakan struktur data *Set* atau *Map* ($O(n)$) untuk mencapai performa yang lebih baik pada data skala besar.